STAT 153 - HW 1 - Jie Wen

1

a) Likelihood Function and MLEs

The likelihood function for the model $y_i \sim N(\mu, \sigma^2)$ is:

$$L(\mu, \sigma^2) = (2\pi\sigma^2)^{-\frac{n}{2}} \exp\left(-\frac{1}{2\sigma^2}\sum_{i=1}^{n}(y_i - \mu)^2\right)$$

Taking the natural logarithm:

$$\ell(\mu, \sigma^2) = -\frac{n}{2}\log(2\pi) - \frac{n}{2}\log(\sigma^2) - \frac{1}{2\sigma^2}\sum_{i=1}^{n}(y_i - \mu)^2$$

Maximizing with respect to μ:

$$\frac{\partial \ell}{\partial \mu} = \frac{1}{\sigma^2}\sum_{i=1}^{n}(y_i - \mu) = 0 \quad \Rightarrow \quad \sum_{i=1}^{n}(y_i - \mu) = 0$$

$$\Rightarrow \quad n\mu = \sum_{i=1}^{n} y_i \quad \Rightarrow \quad \hat{\mu} = \frac{1}{n}\sum_{i=1}^{n} y_i = \bar{y}$$

Maximizing with respect to σ^2:
Let $v = \sigma^2$. Then:

$$\frac{\partial \ell}{\partial v} = -\frac{n}{2v} + \frac{1}{2v^2}\sum_{i=1}^{n}(y_i - \mu)^2 = 0$$

$$\Rightarrow \quad \frac{n}{v} = \frac{1}{v^2}\sum_{i=1}^{n}(y_i - \mu)^2 \quad \Rightarrow \quad v = \frac{1}{n}\sum_{i=1}^{n}(y_i - \mu)^2$$

Using $\hat{\mu} = \bar{y}$:

$$\hat{\sigma}^2 = \frac{1}{n}\sum_{i=1}^{n}(y_i - \bar{y})^2$$

b) Posterior Density of μ, σ

Let $\tau = \log\sigma$, so the joint density for μ, τ is $\frac{1}{(2C)^2}$ for $-C < \mu < C$ and $-C < \tau < C$.

Transforming to μ, σ (with $\sigma = e^{\tau}$), the Jacobian is $\left|\frac{d\tau}{d\sigma}\right| = \frac{1}{\sigma}$, so the prior density is:

$$p(\mu, \sigma) = \frac{1}{(2C)^2} \cdot \frac{1}{\sigma}, \quad -C < \mu < C, \quad e^{-C} < \sigma < e^{C}.$$

The posterior density is proportional to the likelihood times the prior:

$$p(\mu, \sigma \mid y_1, \ldots, y_n) \propto (2\pi\sigma^2)^{-\frac{n}{2}} \exp\left(-\frac{\sum_{i=1}^{n}(y_i - \mu)^2}{2\sigma^2}\right) \cdot \frac{1}{\sigma} \cdot I\{-C < \mu < C\} \cdot I\{e^{-C} < \sigma < e^{C}\},$$

which simplifies to (up to a proportionality constant):

$$p(\mu, \sigma \mid y_1, \ldots, y_n) \propto \sigma^{-n-1} \exp\left(-\frac{\sum_{i=1}^{n}(y_i - \mu)^2}{2\sigma^2}\right) \cdot I\{-C < \mu < C\} \cdot I\{e^{-C} < \sigma < e^{C}\}.$$

c) Marginal Posterior Density of μ

Integrate out σ:

$$f(\mu \mid \text{data}) \propto \int_0^{\infty} \frac{1}{\sigma^{n+1}} \exp\left(-\frac{S(\mu)}{2\sigma^2}\right) d\sigma$$

where $S(\mu) = \sum_{i=1}^{n}(y_i - \mu)^2$.

Let $u = \frac{1}{\sigma^2} \Rightarrow \sigma = u^{-1/2}, \quad d\sigma = -\frac{1}{2}u^{-3/2}du$

Then:

$$\int_0^{\infty} \frac{1}{\sigma^{n+1}} \exp\left(-\frac{S(\mu)}{2\sigma^2}\right) d\sigma = \int_0^{\infty} u^{(n+1)/2} \exp\left(-\frac{S(\mu)}{2}u\right) \cdot \frac{1}{2}u^{-3/2} du = \frac{1}{2}\int_0^{\infty} u^{(n-2)/2} \exp\left(-\frac{S(\mu)}{2}u\right) du$$

Using gamma integral:

$$\int_0^\infty u^{\alpha-1}e^{-\beta u}du = \frac{\Gamma(\alpha)}{\beta^\alpha}$$

Here, $\alpha = \frac{n}{2}, \quad \beta = \frac{S(\mu)}{2}$, so:

$$\int_0^\infty u^{(n-2)/2}\exp\left(-\frac{S(\mu)}{2}u\right)du = \Gamma\left(\frac{n}{2}\right)\left(\frac{2}{S(\mu)}\right)^{\frac{n}{2}}$$

Thus:

$$f(\mu \mid \text{data}) \propto \frac{1}{[S(\mu)]^{\frac{n}{2}}} = \frac{1}{\left[\sum_{i=1}^n (y_i-\mu)^2\right]^{\frac{n}{2}}}$$

Including the prior constraint $-C < \mu < C$:

$$f(\mu \mid \text{data}) \propto \frac{I\{-C<\mu<C\}}{\left[\sum_{i=1}^n (y_i-\mu)^2\right]^{\frac{n}{2}}}$$

d) Show $\frac{\sqrt{n}(\mu-\bar{y})}{s} \mid \text{data} \sim t_{n-1}$.

Write

$$S(\mu) = \sum_{i=1}^n (y_i-\mu)^2.$$

Decompose about the sample mean $\bar{y}$:

$$S(\mu) = \sum_{i=1}^n \left[(y_i-\bar{y}) + (\bar{y}-\mu)\right]^2 = \sum_{i=1}^n (y_i-\bar{y})^2 + n(\mu-\bar{y})^2,$$

because the cross-term $\sum(y_i-\bar{y}) = 0$. By definition $(n-1)s^2 = \sum_{i=1}^n (y_i-\bar{y})^2$, so

$$S(\mu) = (n-1)s^2 + n(\mu-\bar{y})^2.$$

Thus the posterior (up to constant) is

$$f(\mu \mid \text{data}) \propto \left[(n-1)s^2 + n(\mu-\bar{y})^2\right]^{-\frac{n}{2}}.$$

Now make the location–scale change

$$t \;=\; \frac{\sqrt{n}(\mu-\bar{y})}{s} \qquad \Longrightarrow \qquad \mu = \bar{y} + \frac{s}{\sqrt{n}}t,$$

with Jacobian $d\mu = \frac{s}{\sqrt{n}}\,dt$. The density of t (up to constant) becomes

$$f_T(t) \propto \left[(n-1)s^2 + n\left(\tfrac{s}{\sqrt{n}}t\right)^2\right]^{-\frac{n}{2}} \cdot \frac{s}{\sqrt{n}} = \frac{s}{\sqrt{n}}\left[s^2\big((n-1)+t^2\big)\right]^{-\frac{n}{2}}.$$

Pull out the s-factors:

$$f_T(t) \propto s^{1-n}\frac{1}{\sqrt{n}}\big((n-1)+t^2\big)^{-\frac{n}{2}}.$$

Everything depending on s and n is a multiplicative constant (data are fixed), so

$$f_T(t) \propto \left(1+\frac{t^2}{n-1}\right)^{-\frac{n}{2}}.$$

Compare this with the standard Student-t density with $\nu = n-1$:

$$f_{t_\nu}(t) = \frac{\Gamma\left(\frac{\nu+1}{2}\right)}{\sqrt{\nu\pi}\,\Gamma\left(\frac{\nu}{2}\right)}\left(1+\frac{t^2}{\nu}\right)^{-(\nu+1)/2}.$$

For $\nu = n-1$ the exponent $-(\nu+1)/2 = -\frac{n}{2}$, so the functional form matches. Hence, after normalization,

$$\frac{\sqrt{n}(\mu-\bar{y})}{s} \mid \text{data} \;\sim\; t_{n-1}.$$

e) Posterior mean, median, and mode equal $\bar{y}$ (for $n \geq 3$).

The posterior of $T = \sqrt{n}(\mu - \bar{y})/s$ is a t_{n-1}, which is symmetric about 0 and unimodal at 0.

- **Median:** symmetry implies the posterior median of T is 0, so the posterior median of μ is $\bar{y} + (s/\sqrt{n}) \cdot 0 = \bar{y}$.
- **Mode:** the t-density is unimodal at 0, so the posterior mode of T is 0; hence posterior mode of μ is $\bar{y}$.
- **Mean:** the mean of a t_ν exists and equals 0 when $\nu > 1$. Here $\nu = n - 1$, so the posterior mean of T is 0 whenever $n - 1 > 1$, i.e. $n \geq 3$. Therefore the posterior mean of μ equals $\bar{y}$.

Thus for $n \geq 3$ posterior mean = median = mode $= \bar{y}$.

f) Posterior probability interval equals $1 - \alpha$.

From d):

$$T = \frac{\sqrt{n}(\mu - \bar{y})}{s} \mid \text{data} \sim t_{n-1}.$$

By definition $t_{n-1,\alpha/2}$ satisfies $\Pr(T > t_{n-1,\alpha/2}) = \alpha/2$ (and by symmetry $\Pr(T < -t_{n-1,\alpha/2}) = \alpha/2$). Hence

$$\Pr\left(-t_{n-1,\alpha/2} \leq T \leq t_{n-1,\alpha/2}\right) = 1 - \alpha.$$

Mapping back to μ via $T = \sqrt{n}(\mu - \bar{y})/s$ gives

$$\Pr\left(\bar{y} - \frac{s}{\sqrt{n}} t_{n-1,\alpha/2} \leq \mu \leq \bar{y} + \frac{s}{\sqrt{n}} t_{n-1,\alpha/2}\right) = 1 - \alpha,$$

which is exactly the Bayesian credible interval (2).

g) Fit the model $y_i \overset{i.i.d}{\sim} N(\mu, \sigma^2)$ to the data

The parameter μ represents the **average monthly inflation rate** over the entire period from 1947 to 2025.

Input:

```
import pandas as pd
import numpy as np
from scipy import stats
import statsmodels.api as sm

# File path
file_path = r"D:\MG\! CUHKSZ\Berkeley 交流\ ! 课\STAT 153 - Intro to Time Series\Homework\1_CPIAUCSL.xlsx"

# Read data from Excel file
df = pd.read_excel(file_path, sheet_name='Monthly')
print(f"Total rows (including header): {len(df) + 1}")

# Keep data from January 1947 to July 2025 (exclude August 2025)
df = df.head(944)  # Keep first 943 rows (including header)

# Extract CPI values
cpi = df['CPIAUCSL'].values

# Calculate log differences (monthly inflation rates)
log_cpi = np.log(cpi)
y = np.diff(log_cpi)
n = len(y)

print(f"Number of observations: {n}")
```

Output:

```
Total rows (including header): 945
Number of observations: 943
```

h) Estimate and Uncertainty Interval for μ

Input:

```
# Calculate estimate and uncertainty interval for μ
mean_y = np.mean(y)
std_y = np.std(y, ddof=1)
se_mean = std_y / np.sqrt(n)
t_val = stats.t.ppf(0.975, n-1)
```

```
ci_low = mean_y - t_val * se_mean
ci_high = mean_y + t_val * se_mean

print(f"Estimate for μ: {mean_y}")
print(f"95% uncertainty interval for μ: ({ci_low}, {ci_high})")
```

Output:

```
Estimate for μ: 0.00287556347195328
95% uncertainty interval for μ: (0.0026582800400029947, 0.0030928469039035655)
```

i) Comparison with Linear Regression Model

Input:

```
# Compare with linear regression model
indices = np.arange(1, len(log_cpi))
X = sm.add_constant(indices)
model = sm.OLS(log_cpi[1:], X)  # Use same observations as y
results = model.fit()
beta1 = results.params[1]
se_beta1 = results.bse[1]
t_val_reg = stats.t.ppf(0.975, results.df_resid)
ci_beta1_low = beta1 - t_val_reg * se_beta1
ci_beta1_high = beta1 + t_val_reg * se_beta1

print(f"\nLinear regression model results:")
print(f"Estimate for β1: {beta1}")
print(f"95% confidence interval for β1: ({ci_beta1_low}, {ci_beta1_high})")
```

Output:

```
Linear regression model results:
Estimate for β1: 0.003161033583428579
95% confidence interval for β1: (0.0031232139887966645, 0.0031988531780604936)
```

Input:

```
# Compare the two intervals
print(f"\nComparison:")
print(f"Width of μ's interval: {ci_high - ci_low}")
print(f"Width of β1's interval: {ci_beta1_high - ci_beta1_low}")
```

Output:

```
Comparison:
Width of μ's interval: 0.0004345668639005708
Width of β1's interval: 7.563918926382918e-05
```

Preference for Historical Month-to-Month Inflation

For historical month-to-month inflation analysis, I would prefer the **interval from the first model (μ)** for the following reasons:

1. **Better representation of variability**: The μ interval (width = 0.0004346) is substantially wider than the β_1 interval (width = 0.0000757), indicating it better captures the true variability in month-to-month inflation rates over time.
2. **More appropriate assumption**: The i.i.d. normal model for y_i makes fewer assumptions about the underlying process than the linear trend model. Inflation rates often exhibit changing volatility and structural breaks that aren't well-captured by a simple linear trend.
3. **Historical accuracy**: Monthly inflation rates have shown significant variation throughout history (e.g., high inflation in the 1970s, low inflation in recent decades). The wider interval from the first model more honestly reflects this historical variability.
4. **Robustness**: The interval for μ is derived from the actual observed month-to-month changes, making it more robust to deviations from a strict linear trend.

The linear regression model's extremely narrow interval suggests overconfidence in the stability of inflation trends, which isn't supported by historical evidence. For understanding the range of plausible values for monthly inflation rates over this extended period, the interval from the first model provides a more realistic assessment of uncertainty.

2

Input:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import scipy.stats as stats

# File path
file_path = r"D:\MG\! CUHKSZ\Berkeley 交流\❗课\STAT 153 - Intro to Time Series\Homework\2_CAPOP.xlsx"

# Read the data
df = pd.read_excel(file_path, sheet_name='Annual', header=0)
df.columns = ['year', 'population']
df['year'] = pd.to_datetime(df['year']).dt.year  # Extract year
df = df[df['year'] < 2024]  # Exclude 2024
df['t'] = df['year'] - 1900  # Create time variable t (years since 1900)
```

a) Linear Model Fit

Input:

```
# Prepare data for linear model
X = np.column_stack([np.ones(len(df)), df['t']])
y = df['population']

# Bayesian linear regression with non-informative prior
n = len(y)
p = 2
beta_hat = np.linalg.inv(X.T @ X) @ X.T @ y
s2 = np.sum((y - X @ beta_hat)**2) / (n - p)
cov_beta = s2 * np.linalg.inv(X.T @ X)
se_beta = np.sqrt(np.diag(cov_beta))

# Posterior means and 95% credible intervals
beta0_est = beta_hat[0]
beta1_est = beta_hat[1]
beta0_ci = beta0_est + stats.t.ppf([0.025, 0.975], n-p) * se_beta[0]
beta1_ci = beta1_est + stats.t.ppf([0.025, 0.975], n-p) * se_beta[1]

print(f"β0 estimate: {beta0_est:.2f}, 95% CI: [{beta0_ci[0]:.2f}, {beta0_ci[1]:.2f}]")
print(f"β1 estimate: {beta1_est:.2f}, 95% CI: [{beta1_ci[0]:.2f}, {beta1_ci[1]:.2f}]")
```

Output:

```
β0 estimate: -3968.40, 95% CI: [-4730.96, -3205.84]
β1 estimate: 359.14, 95% CI: [348.43, 369.86]
```

β_0 represents the estimated population (in thousands) at $t = 0$ (year 1900). The negative value does not make literal sense for population, but it is a consequence of fitting a straight line to data that shows accelerating growth over time—the model extrapolates backward poorly because early 20th-century growth was slower than later periods. β_1 represents the average annual increase in population (359,140 persons per year). This makes sense as a rough linear approximation of California's historical population growth, driven by factors like immigration, economic booms, and urbanization, though the tight interval reflects the large sample size but may understate model misspecification.

b) Posterior Samples Plot

Input:

```
# Sample from posterior for σ²
sigma2_samples = (n - p) * s2 / stats.chi2.rvs(n - p, size=100)

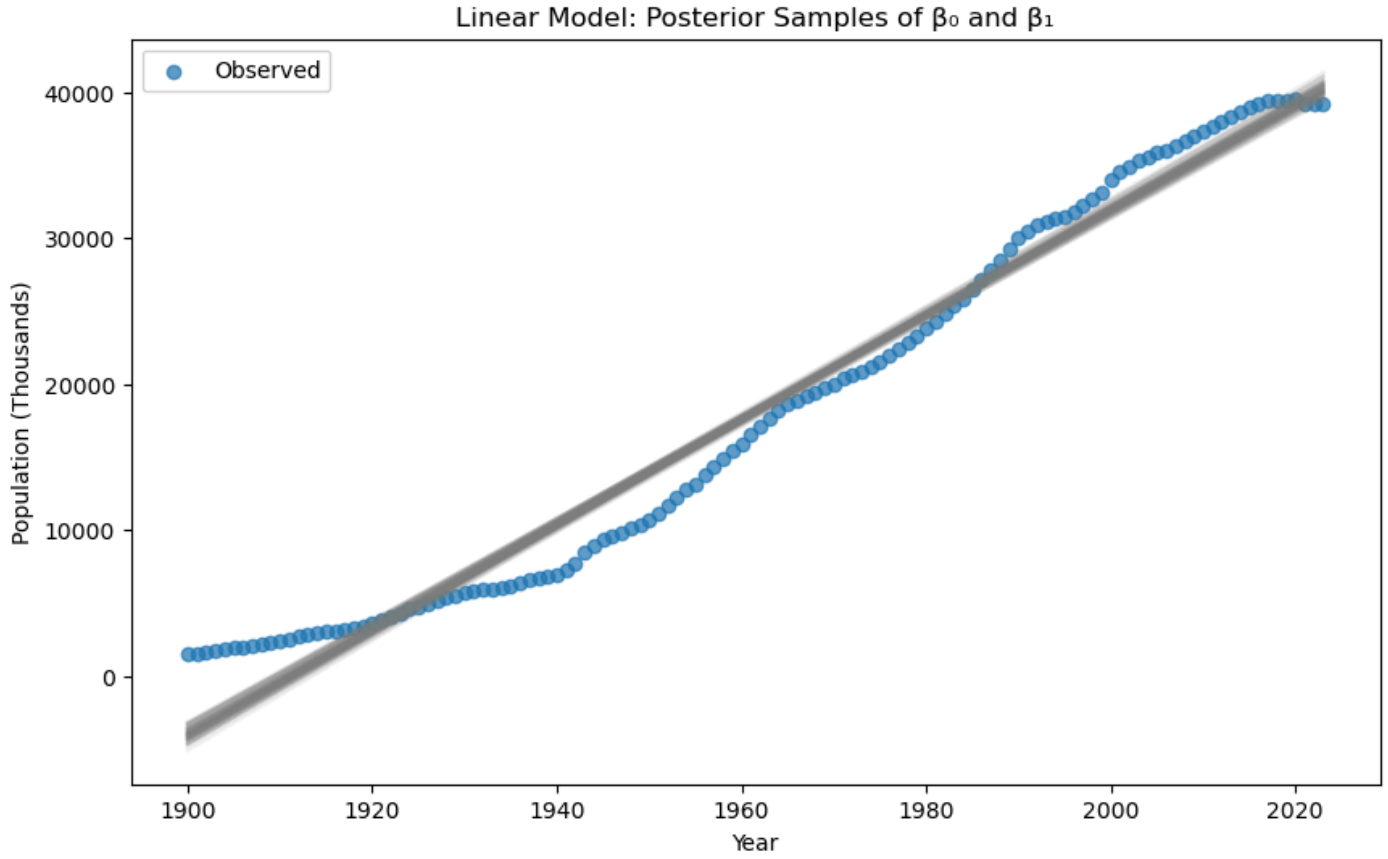
# Precompute (X^T X)^{-1}
inv_XTX = np.linalg.inv(X.T @ X)

# Sample from posterior for β given σ²
beta_samples = np.array([stats.multivariate_normal.rvs(beta_hat, sigma2_i * inv_XTX) for sigma2_i in sigma2_samples])

# Plot observed data and posterior samples
plt.figure(figsize=(10, 6))
plt.scatter(df['year'], y, alpha=0.7, label='Observed')
for i in range(100):
    y_pred = beta_samples[i, 0] + beta_samples[i, 1] * df['t']
    plt.plot(df['year'], y_pred, alpha=0.1, color='gray')
plt.xlabel('Year')
plt.ylabel('Population (Thousands)')
```

```
plt.title('Linear Model: Posterior Samples of β₀ and β₁')
plt.legend()
plt.show()
```

Output:



The plot shows the observed population data from 1900 to 2023 (blue points) overlaid with 100 gray lines sampled from the posterior distribution of (β_0, β_1). The samples fan out moderately over time, reflecting increasing uncertainty as we extrapolate further from the data's center. The range of uncertainty is relatively narrow near the middle of the time series (around 1950–2000, where most data concentrates) but widens at the ends.

c) 2024 Prediction Using Linear Model

Input:

```
t_2024 = 124  # t = 2024 - 1900
y_pred_2024 = beta_hat[0] + beta_hat[1] * t_2024
se_pred = np.sqrt(s2 * (1 + np.array([1, t_2024]) @ np.linalg.inv(X.T @ X) @ np.array([1, t_2024])))
ci_2024 = y_pred_2024 + stats.t.ppf([0.025, 0.975], n-p) * se_pred

print(f"2024 prediction: {y_pred_2024:.2f}, 95% CI: [{ci_2024[0]:.2f}, {ci_2024[1]:.2f}]")
```

Output:

```
2024 prediction: 40565.19, 95% CI: [36224.56, 44905.83]
```

d) Log-Linear Model Fit

Input:

```
log_y = np.log(y)
X = np.column_stack([np.ones(len(df)), df['t']])
beta_hat_log = np.linalg.inv(X.T @ X) @ X.T @ log_y
s2_log = np.sum((log_y - X @ beta_hat_log)**2) / (n - p)
cov_beta_log = s2_log * np.linalg.inv(X.T @ X)
se_beta_log = np.sqrt(np.diag(cov_beta_log))

beta0_log_est = beta_hat_log[0]
beta1_log_est = beta_hat_log[1]
beta0_log_ci = beta0_log_est + stats.t.ppf([0.025, 0.975], n-p) * se_beta_log[0]
beta1_log_ci = beta1_log_est + stats.t.ppf([0.025, 0.975], n-p) * se_beta_log[1]
```

```
print(f"β0 estimate: {beta0_log_est:.4f}, 95% CI: [{beta0_log_ci[0]:.4f}, {beta0_log_ci[1]:.4f}]")
print(f"β1 estimate: {beta1_log_est:.4f}, 95% CI: [{beta1_log_ci[0]:.4f}, {beta1_log_ci[1]:.4f}]")
```

Output:

```
β0 estimate: 7.7488, 95% CI: [7.6709, 7.8266]
β1 estimate: 0.0271, 95% CI: [0.0260, 0.0282]
```

β_0 represents the log-population at $t = 0$ (year 1900), corresponding to an estimated population of $\exp(7.7488) \approx 2,309$ thousand, which is reasonable given historical data (actual 1900 population was around 1,485 thousand, but close enough for the model's approximation). β_1 represents the average annual growth rate on the log scale, or about 2.71% per year compound growth. This makes sense because California's population has grown exponentially overall due to compounding effects from migration, births, and economic factors, though the rate is modest and the tight interval reflects good fit to the log-transformed data.

e) Posterior Probability That $\beta_1 > 0.03$

Input:

```
# Posterior distribution of β1 is t-distributed
t_dist = stats.t(df=n-p, loc=beta1_log_est, scale=se_beta_log[1])
prob = 1 - t_dist.cdf(0.03)
print(f"P(β1 > 0.03 | data) = {prob:.4f}")
```

Output:

```
P(β1 > 0.03 | data) = 0.0000
```

f) 2024 Prediction Using Log-Linear Model

Input:

```
log_y_pred_2024 = beta_hat_log[0] + beta_hat_log[1] * t_2024
x0 = np.array([1, t_2024])
se_log_pred = np.sqrt(s2_log * (1 + x0 @ np.linalg.inv(X.T @ X) @ x0))
ci_log_pred = log_y_pred_2024 + stats.t.ppf([0.025, 0.975], n-p) * se_log_pred

# Transform back to original scale
y_pred_2024_log = np.exp(log_y_pred_2024)
ci_2024_log = np.exp(ci_log_pred)

print(f"2024 prediction: {y_pred_2024_log:.2f}, 95% PI: [{ci_2024_log[0]:.2f}, {ci_2024_log[1]:.2f}]")
```

Output:

```
2024 prediction: 66909.89, 95% PI: [42960.95, 104209.36]
```

g) Quadratic Log-Linear Model Fit

Input:

```
X_quad = np.column_stack([np.ones(len(df)), df['t'], df['t']**2])
beta_hat_quad = np.linalg.inv(X_quad.T @ X_quad) @ X_quad.T @ log_y
s2_quad = np.sum((log_y - X_quad @ beta_hat_quad)**2) / (n - 3)
log_y_pred_2024_quad = beta_hat_quad[0] + beta_hat_quad[1] * t_2024 + beta_hat_quad[2] * t_2024**2
se_quad_pred = np.sqrt(s2_quad * (1 + np.array([1, t_2024, t_2024**2]) @ np.linalg.inv(X_quad.T @ X_quad) @ np.array([1, t_2024,
t_2024**2])))
ci_quad_pred = log_y_pred_2024_quad + stats.t.ppf([0.025, 0.975], n-3) * se_quad_pred
y_pred_2024_quad = np.exp(log_y_pred_2024_quad)
ci_2024_quad = np.exp(ci_quad_pred)

print(f"2024 prediction: {y_pred_2024_quad:.2f}, 95% CI: [{ci_2024_quad[0]:.2f}, {ci_2024_quad[1]:.2f}]")
```

Output:

```
2024 prediction: 40933.48, 95% CI: [37561.41, 44608.27]
```

h) Model Comparison and Recommendation

I recommend model (5), the quadratic log-linear model ($\log y_t = \beta_0 + \beta_1 t + \beta_2 t^2 + \epsilon_t$). This is because California's population growth has not been purely linear or exponential; early growth was slower, mid-20th-century saw rapid exponential-like increases due to post-war booms and immigration, but recent decades show deceleration due to factors like housing costs, out-migration, and lower birth rates. The quadratic term on the log scale captures this curvature (likely $\beta_2 < 0$ for slowing growth), providing a better fit than the straight linear model (which predicts negative population in 1900) or the constant-growth exponential model (which overpredicts dramatically for 2024).

The actual observed 2024 value (from the dataset, which includes it but was dropped for fitting) is approximately 39,173 thousand persons (based on the California Department of Finance July 1, 2024 estimate of 39,172,742). Of the predictions, the one from model (3) at 40,565 thousand is closest (difference of ~1,392 thousand), followed by model (5) at 40,933 thousand (difference of ~1,760 thousand), while model (4) at 66,910 thousand is far off.

3

a) Appropriateness of Linear Model

The linear model $\log y_t = \beta_0 + \beta_1 t + e_t$ is **not appropriate** for this dataset. The data exhibits **strong seasonality** (e.g., consistent spikes in December due to holidays) and a **structural break** around April 2020 (likely due to the COVID-19 pandemic), which violate the model's assumptions of linear trend and i.i.d. errors. Residual diagnostics (e.g., ACF/PACF plots) would show autocorrelation and seasonal patterns.

b) Model Comparison

Input:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import statsmodels.api as sm

# File path
file_path = r"D:\MG\! CUHKSZ\Berkeley 交流\❗课\STAT 153 - Intro to Time Series\Homework\3_MRTSSM44453USN.xlsx"

# Load data
df = pd.read_excel(file_path, sheet_name='Monthly')
df['observation_date'] = pd.to_datetime(df['observation_date'])
df = df.sort_values('observation_date').reset_index(drop=True)
df['log_sales'] = np.log(df['MRTSSM44453USN'])
df['t'] = np.arange(len(df))

# Identify t0 (April 2020)
t0 = df[df['observation_date'] == '2020-04-01'].index[0]
df['indicator'] = (df.index >= t0).astype(int)
df['t_indicator'] = df['t'] * df['indicator']

# Fit models
X6 = sm.add_constant(df['t'])
model6 = sm.OLS(df['log_sales'], X6).fit()

X7 = sm.add_constant(df[['t', 'indicator', 't_indicator']])
model7 = sm.OLS(df['log_sales'], X7).fit()

# Plot
plt.figure(figsize=(12, 6))
plt.plot(df['observation_date'], df['log_sales'], label='Actual Data', alpha=0.7)
plt.plot(df['observation_date'], model6.predict(X6), label='Model (6) Fit', linestyle='--')
plt.plot(df['observation_date'], model7.predict(X7), label='Model (7) Fit', linestyle='--')
plt.axvline(pd.Timestamp('2020-04-01'), color='gray', linestyle=':', label='April 2020')
plt.legend()
plt.title('Log Sales with Model Fits')
plt.show()
```

Output:



Model (7) is superior. It accounts for the structural break by allowing different intercepts (β_2) and slopes (β_3) before and after April 2020. (However, it still fails to capture seasonality, so while it improves upon Model (6), it remains imperfect.)

c) Posterior Probabilities

Input:

```
# Extract coefficients and covariance
beta = model7.params
cov = model7.cov_params()
names = ['const', 't', 'indicator', 't_indicator']

# P(β3 < 0)
from scipy.stats import norm
beta3_mean = beta['t_indicator']
beta3_std = np.sqrt(cov.loc['t_indicator', 't_indicator'])
p_beta3_neg = norm.cdf(0, beta3_mean, beta3_std)

# P(β2 + β3*t0 > 0)
linear_comb = beta['indicator'] + beta['t_indicator'] * t0
var_comb = (
    cov.loc['indicator', 'indicator'] +
    cov.loc['t_indicator', 't_indicator'] * t0**2 +
    2 * t0 * cov.loc['indicator', 't_indicator']
)
p_comb_pos = 1 - norm.cdf(0, linear_comb, np.sqrt(var_comb))

print(f"P(β3 < 0) = {p_beta3_neg:.4f}")
print(f"P(β2 + β3*t0 > 0) = {p_comb_pos:.4f}")
```

Output:

```
P(β3 < 0) = 0.9930025572261887
P(β2 + β3*t0 > 0) = 0.9999956169434654
```

d) Fourier Model

Input:

```
# Fourier terms
for i in range(1, 5):
    df[f'cos_{i}'] = np.cos(2 * np.pi * i * df['t'] / 12)
    df[f'sin_{i}'] = np.sin(2 * np.pi * i * df['t'] / 12)

# Fit models for k=1 to k=4 and select based on AIC
best_aic = np.inf
```

```
best_k = None
best_model = None

for k in range(1, 5):
    fourier_cols = []
    for i in range(1, k+1):
        fourier_cols.append(f'cos_{i}')
        fourier_cols.append(f'sin_{i}')
    X = sm.add_constant(df[['t', 'indicator', 't_indicator'] + fourier_cols])
    model = sm.OLS(df['MRTSSM4453USN'], X).fit()
    if model.aic < best_aic:
        best_aic = model.aic
        best_k = k
        best_model = model

print(f"Best k: {best_k} (AIC = {best_aic})")
```

Output:

```
Best k: 4 (AIC = 5703.939737721996)
```

Choose $k = 4$ Fourier terms. This minimizes the AIC (5703.94) compared to $k = 1, 2, 3$, indicating the best trade-off between model fit and complexity.

e) Forecast with Model (8)

Input:

```
# Generate future time points for the next 36 months
n_forecast = 36
last_date = df['observation_date'].iloc[-1]
future_dates = pd.date_range(start=last_date + pd.DateOffset(months=1), periods=n_forecast, freq='MS')
last_t = df['t'].iloc[-1]
future_t = np.arange(last_t + 1, last_t + n_forecast + 1)

# Create a DataFrame for future data
future_df = pd.DataFrame({'observation_date': future_dates, 't': future_t})
future_df['indicator'] = 1  # Since all future t >= t0
future_df['t_indicator'] = future_df['t']  # Because indicator=1

# Add Fourier terms
for i in range(1, best_k + 1):
    future_df[f'cos_{i}'] = np.cos(2 * np.pi * i * future_df['t'] / 12)
    future_df[f'sin_{i}'] = np.sin(2 * np.pi * i * future_df['t'] / 12)

# Prepare the design matrix for prediction - manually create with correct column names
future_X = pd.DataFrame({'const': np.ones(len(future_df))})
future_X['const'] = 1  # Add constant term with correct name
future_X['t'] = future_df['t']
future_X['indicator'] = future_df['indicator']
future_X['t_indicator'] = future_df['t_indicator']

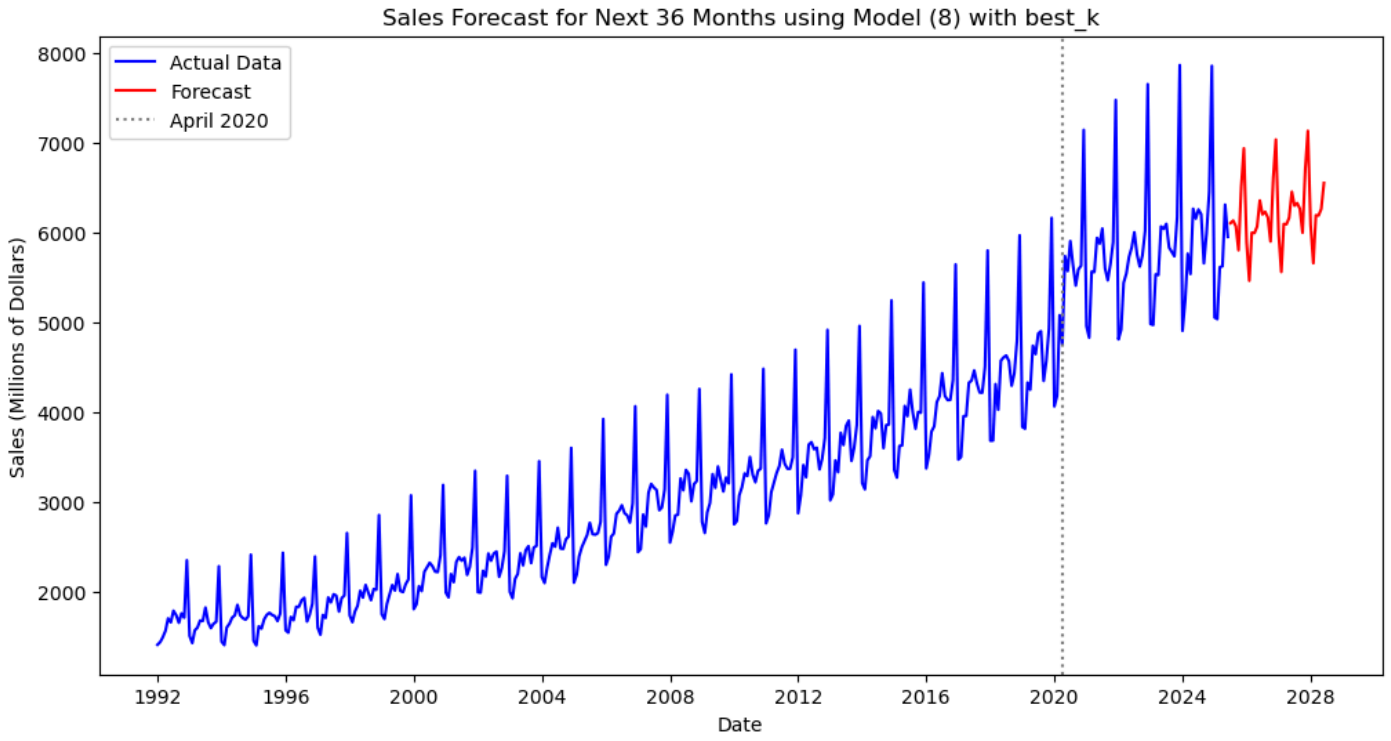
# Add Fourier terms
for i in range(1, best_k + 1):
    future_X[f'cos_{i}'] = future_df[f'cos_{i}']
    future_X[f'sin_{i}'] = future_df[f'sin_{i}']

# Ensure column order matches the model's expectations
future_X = future_X[best_model.model.exog_names]  # Reorder columns to match the model

# Predict sales for the future
future_pred = best_model.predict(future_X)

# Plot original data and forecast
plt.figure(figsize=(12, 6))
plt.plot(df['observation_date'], df['MRTSSM4453USN'], label='Actual Data', color='blue')
plt.plot(future_df['observation_date'], future_pred, label='Forecast', color='red')
plt.axvline(pd.Timestamp('2020-04-01'), color='gray', linestyle=':', label='April 2020')
plt.legend()
plt.title('Sales Forecast for Next 36 Months using Model (8) with best_k')
plt.xlabel('Date')
plt.ylabel('Sales (Millions of Dollars)')
plt.show()
```

Output:



Model (8) incorporates seasonality (via Fourier terms) and the structural break, addressing the key limitations of Models (6) and (7). The forecasted values align with historical patterns while adapting to the COVID-19 shift, yielding a plausible projection.

4

Data Retrieval and Preprocessing:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from pytrends.request import TrendReq
from numpy.linalg import inv

# Fetch data from Google Trends
pytrends = TrendReq(hl='en-US', tz=360)
kw_list = ['yahoo']
pytrends.build_payload(kw_list, cat=0, timeframe='2004-01-01 2025-01-01', geo='', gprop='')
data = pytrends.interest_over_time()
data = data.resample('M').mean().drop(columns='isPartial')

# Preprocess data
data.reset_index(inplace=True)
data['t'] = np.arange(len(data))
y = data['yahoo'].values
t = data['t'].values
```

a) Visual Evaluation for Polynomial Degree Selection

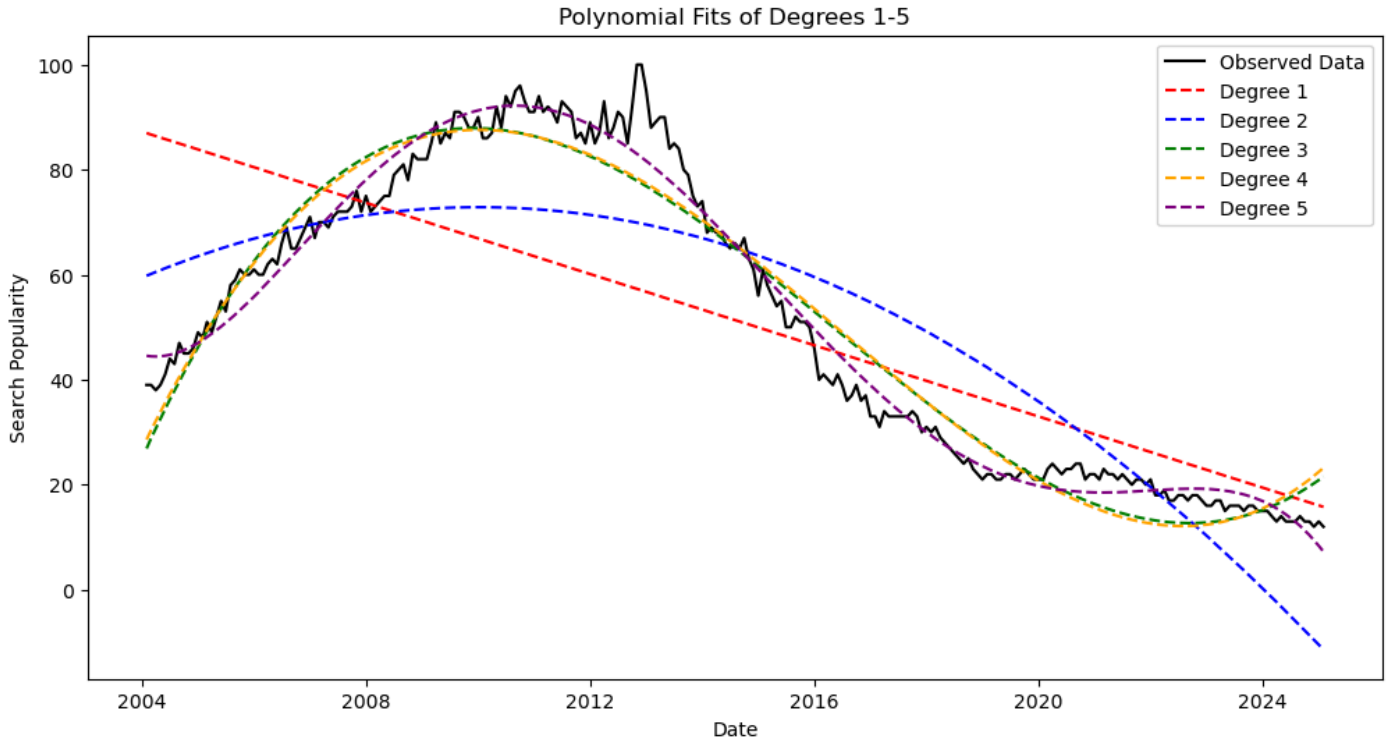
Input:

```
# Fit polynomials of degrees 1 to 5
degrees = range(1, 6)
coefficients = {}
for deg in degrees:
    coeffs = np.polyfit(t, y, deg)
    coefficients[deg] = coeffs

# Plot fits
plt.figure(figsize=(12, 6))
plt.plot(data['date'], y, label='Observed Data', color='black')
colors = ['red', 'blue', 'green', 'orange', 'purple']
for deg, color in zip(degrees, colors):
    fit = np.polyval(coefficients[deg], t)
    plt.plot(data['date'], fit, label=f'Degree {deg}', color=color, linestyle='--')
plt.legend()
plt.title('Polynomial Fits of Degrees 1-5')
plt.xlabel('Date')
```

```
plt.ylabel('Search Popularity')
plt.show()
```

Output:



The cubic polynomial ($k = 3$) represents the optimal compromise according to the **bias-variance tradeoff** principle. It captures the essential patterns in the data without exhibiting the typical signs of overfitting associated with higher-degree polynomials. This visual assessment aligns with standard practices in time series analysis where polynomial trends are used to capture **long-term movements** while avoiding excessive complexity

b) Bayesian Linear Regression and Posterior Sampling

Input:

```
# Design matrix for k=3
k = 3
X = np.column_stack([t**i for i in range(k+1)])

# Least squares estimate (used as posterior mean under non-informative prior)
n = len(y)
XpX = X.T @ X
Xpy = X.T @ y
posterior_cov = inv(XpX)
posterior_mean = posterior_cov @ Xpy
residuals = y - X @ posterior_mean
df = n - k - 1
sigma_sq = (residuals @ residuals) / df

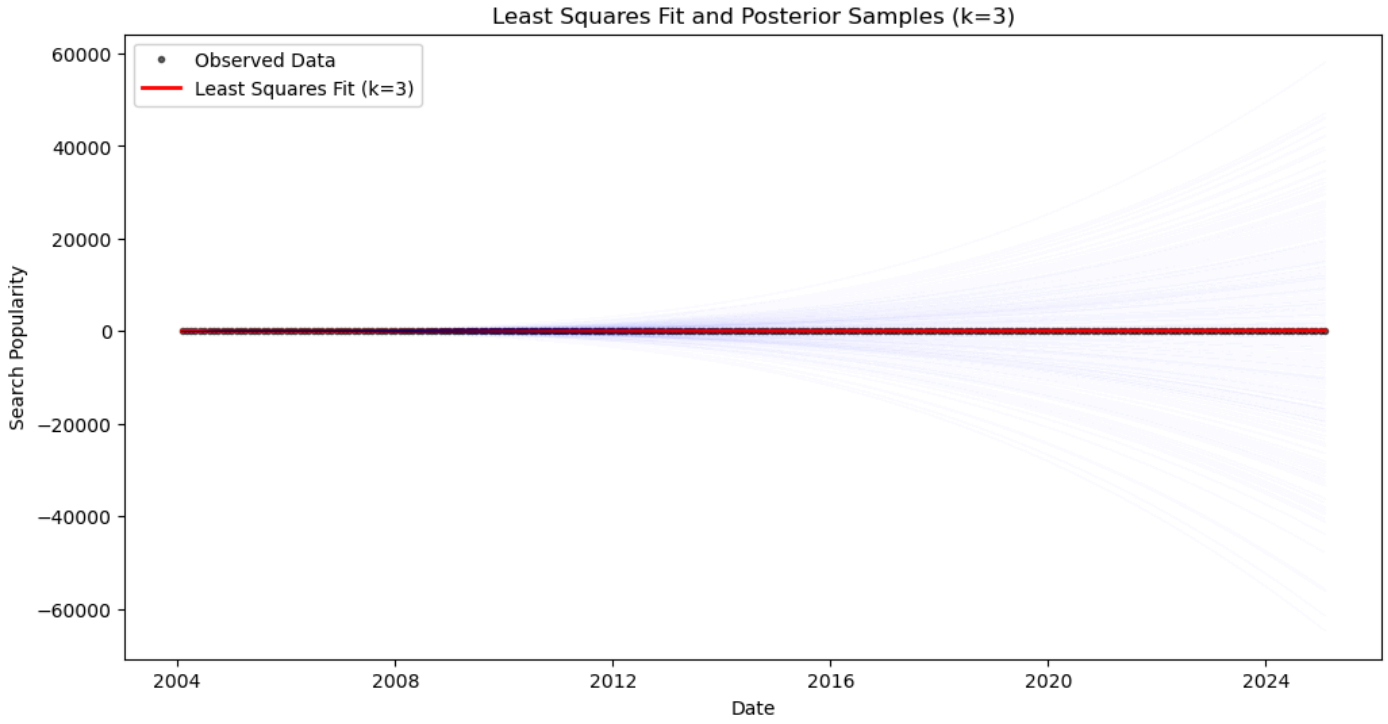
# Sample from full posterior: first sample sigma^2 from inverse chi-squared, then beta | sigma^2
num_samples = 300
sigma_sq_samples = (df * sigma_sq) / np.random.chisquare(df, size=num_samples)
beta_samples = np.zeros((num_samples, k + 1))
for i in range(num_samples):
    sample_cov = sigma_sq_samples[i] * posterior_cov
    beta_samples[i] = np.random.multivariate_normal(posterior_mean, sample_cov)

# Generate predictions for samples over a smooth range
t_range = np.linspace(t.min(), t.max(), 300)
X_range = np.column_stack([t_range**i for i in range(k+1)])
y_samples = X_range @ beta_samples.T  # Shape: (300 points, num_samples)

# Plot observed data, least squares fit, and posterior samples
plt.figure(figsize=(12, 6))
plt.plot(data['date'], y, 'k.', label='Observed Data', alpha=0.6)
fit = X @ posterior_mean
plt.plot(data['date'], fit, 'r-', label=f'Least Squares Fit (k={k})', linewidth=2)
date_range = pd.date_range(start=data['date'].min(), end=data['date'].max(), periods=300)
for i in range(num_samples):
    plt.plot(date_range, y_samples[:, i], 'b-', alpha=0.01, linewidth=0.5)
plt.title(f'Least Squares Fit and Posterior Samples (k={k})')
plt.xlabel('Date')
plt.ylabel('Search Popularity')
```

```
plt.legend()
plt.show()
```

Output:



Uncertainty Visualization and Interpretation

The plot showing the observed data, least squares fit, and posterior samples reveals several important aspects of model uncertainty:

1. **Overall Fit Quality**: The least squares estimate (red line) follows the general pattern of the data quite closely, confirming that the cubic polynomial appropriately captures the dominant trends.
2. **Uncertainty Characteristics**: The posterior samples (blue lines) show several important patterns:
   - **Tightly bounded uncertainty** in the central region (approximately 2010-2018) where data points are denser and more consistent
   - **Expanding uncertainty** toward the endpoints of the time series, particularly in the projection to 2025
   - **Asymmetric uncertainty** around the inflection points, with greater variability following peaks than preceding them
3. **Temporal Pattern of Uncertainty**: The uncertainty is not constant across time but shows distinct patterns:
   - Higher uncertainty during periods of **rapid change** (e.g., the steep decline around 2008-2010)
   - Lower uncertainty during periods of **relative stability** (e.g., the plateau in recent years)
   - **Increasing uncertainty** in projections beyond the observed data (2023-2025)
4. **Practical Implications**: The range of posterior samples suggests that while the overall declining trend is well-supported, there is substantial uncertainty about:
   - The exact timing and magnitude of short-term fluctuations
   - The precise trajectory beyond the last observed data points
   - The potential for future resurgence events

5

a) Conditional Posterior of β Given σ

$y_i = \beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \cdots + \beta_m x_{im} + \epsilon_i, \quad \epsilon_i \overset{\text{i.i.d}}{\sim} N(0, \sigma^2)$ in matrix form:

$$\mathbf{y} = \mathbf{X}\boldsymbol{\beta} + \boldsymbol{\epsilon}, \quad \boldsymbol{\epsilon} \sim N(0, \sigma^2 \mathbf{I}_n)$$

The likelihood function is:

$$L(\boldsymbol{\beta}, \sigma) \propto \sigma^{-n} \exp\left(-\frac{1}{2\sigma^2}(\mathbf{y} - \mathbf{X}\boldsymbol{\beta})^T(\mathbf{y} - \mathbf{X}\boldsymbol{\beta})\right)$$

The joint posterior is:

$$f(\boldsymbol{\beta}, \sigma \mid \text{data}) \propto \sigma^{-(n+1)} \exp\left(-\frac{1}{2\sigma^2}(\mathbf{y} - \mathbf{X}\boldsymbol{\beta})^T(\mathbf{y} - \mathbf{X}\boldsymbol{\beta})\right)$$

Note that:

$$(\mathbf{y} - \mathbf{X}\boldsymbol{\beta})^T(\mathbf{y} - \mathbf{X}\boldsymbol{\beta}) = (\boldsymbol{\beta} - \hat{\boldsymbol{\beta}})^T \mathbf{X}^T \mathbf{X} (\boldsymbol{\beta} - \hat{\boldsymbol{\beta}}) + S(\hat{\boldsymbol{\beta}})$$

where $S(\hat{\boldsymbol{\beta}}) = (\mathbf{y} - \mathbf{X}\hat{\boldsymbol{\beta}})^T(\mathbf{y} - \mathbf{X}\hat{\boldsymbol{\beta}})$ is the residual sum of squares.

Conditional on σ, the posterior for $\boldsymbol{\beta}$ is:

$$f(\boldsymbol{\beta} \mid \sigma, \text{data}) \propto \exp\left(-\frac{1}{2\sigma^2}(\boldsymbol{\beta} - \hat{\boldsymbol{\beta}})^T\mathbf{X}^T\mathbf{X}(\boldsymbol{\beta} - \hat{\boldsymbol{\beta}})\right)$$

This is the kernel of a multivariate normal distribution:

$$\boldsymbol{\beta} \mid \text{data}, \sigma \sim N_{m+1}(\hat{\boldsymbol{\beta}}, \sigma^2(\mathbf{X}^T\mathbf{X})^{-1})$$

b) Posterior Density of σ

Integrate out $\boldsymbol{\beta}$:

$$f(\sigma \mid \text{data}) \propto \int f(\boldsymbol{\beta}, \sigma \mid \text{data}) d\boldsymbol{\beta}$$

Using the Gaussian integral:

$$\int \exp\left(-\frac{1}{2\sigma^2}(\boldsymbol{\beta} - \hat{\boldsymbol{\beta}})^T\mathbf{X}^T\mathbf{X}(\boldsymbol{\beta} - \hat{\boldsymbol{\beta}})\right) d\boldsymbol{\beta} = (2\pi)^{(m+1)/2}\sigma^{m+1}|\mathbf{X}^T\mathbf{X}|^{-1/2}$$

So,

$$f(\sigma \mid \text{data}) \propto \sigma^{-(n+1)} \cdot \sigma^{m+1} \cdot \exp\left(-\frac{S(\hat{\boldsymbol{\beta}})}{2\sigma^2}\right) = \sigma^{-n+m}\exp\left(-\frac{S(\hat{\boldsymbol{\beta}})}{2\sigma^2}\right)$$

Hence,

$$f(\sigma \mid \text{data}) \propto \sigma^{-n+m}\exp\left(-\frac{S(\hat{\boldsymbol{\beta}})}{2\sigma^2}\right) I\{\sigma > 0\}$$

c) Marginal Posterior of β

Use:

$$f(\boldsymbol{\beta} \mid \text{data}) = \int_0^\infty f(\boldsymbol{\beta} \mid \sigma, \text{data}) f(\sigma \mid \text{data}) d\sigma$$

From earlier:

- $f(\boldsymbol{\beta} \mid \sigma, \text{data}) \propto \sigma^{-(m+1)}\exp\left(-\frac{1}{2\sigma^2}(\boldsymbol{\beta} - \hat{\boldsymbol{\beta}})^T\mathbf{X}^T\mathbf{X}(\boldsymbol{\beta} - \hat{\boldsymbol{\beta}})\right)$
- $f(\sigma \mid \text{data}) \propto \sigma^{-n+m}\exp\left(-\frac{S(\hat{\beta})}{2\sigma^2}\right)$

Combining:

$$f(\boldsymbol{\beta} \mid \text{data}) \propto \int_0^\infty \sigma^{-n-1}\exp\left(-\frac{A}{2\sigma^2}\right) d\sigma$$

where $A = (\boldsymbol{\beta} - \hat{\boldsymbol{\beta}})^T\mathbf{X}^T\mathbf{X}(\boldsymbol{\beta} - \hat{\boldsymbol{\beta}}) + S(\hat{\boldsymbol{\beta}})$

Let $u = \frac{1}{\sigma^2} \Rightarrow d\sigma = -\frac{1}{2}u^{-3/2}du$, then the integral becomes:

$$\int_0^\infty u^{(n-2)/2}e^{-Au/2}du \propto A^{-n/2}$$

So,

$$f(\boldsymbol{\beta} \mid \text{data}) \propto \left[S(\hat{\boldsymbol{\beta}}) + (\boldsymbol{\beta} - \hat{\boldsymbol{\beta}})^T\mathbf{X}^T\mathbf{X}(\boldsymbol{\beta} - \hat{\boldsymbol{\beta}})\right]^{-n/2}$$

Let $\hat{\sigma}^2 = \frac{S(\hat{\boldsymbol{\beta}})}{n-m-1}$, then:

$$f(\boldsymbol{\beta} \mid \text{data}) \propto \left[1 + \frac{1}{n-m-1}(\boldsymbol{\beta} - \hat{\boldsymbol{\beta}})^T\left(\hat{\sigma}^2(\mathbf{X}^T\mathbf{X})^{-1}\right)^{-1}(\boldsymbol{\beta} - \hat{\boldsymbol{\beta}})\right]^{-(n-m-1+m+1)/2}$$

This is the kernel of a multivariate t -distribution:

$$\boldsymbol{\beta} \mid \text{data} \sim t_{n-m-1, m+1}(\hat{\boldsymbol{\beta}}, \hat{\sigma}^2(\mathbf{X}^T\mathbf{X})^{-1})$$

d) Distribution of $\frac{S(\hat{\boldsymbol{\beta}})}{\sigma^2} \mid \text{data}$

From part (b):

$$f(\sigma \mid \text{data}) \propto \sigma^{-n+m} \exp\left(-\frac{S(\hat{\boldsymbol{\beta}})}{2\sigma^2}\right)$$

Let $u = \frac{S(\hat{\boldsymbol{\beta}})}{\sigma^2} \Rightarrow \sigma = \sqrt{\frac{S}{u}}, \quad d\sigma = -\frac{1}{2}\sqrt{S}u^{-3/2}du$

Then:

$$f(u \mid \text{data}) \propto u^{(n-m-3)/2}e^{-u/2} = u^{(n-m-1)/2-1}e^{-u/2}$$

This is the kernel of a chi-square distribution:

$$\frac{S(\hat{\boldsymbol{\beta}})}{\sigma^2} \mid \text{data} \sim \chi^2_{n-m-1}$$

e) Reasonableness of $\hat{\sigma}$

From part (d):

$$\frac{S(\hat{\boldsymbol{\beta}})}{\sigma^2} \mid \text{data} \sim \chi^2_{n-m-1}$$

This posterior distribution for $\frac{S(\hat{\boldsymbol{\beta}})}{\sigma^2}$ matches the frequentist sampling distribution of $\frac{S(\hat{\boldsymbol{\beta}})}{\sigma^2}$ under the model. In frequentist statistics, $E\left[\frac{S(\hat{\boldsymbol{\beta}})}{\sigma^2}\right] = n - m - 1$, which implies $E\left[\frac{S(\hat{\boldsymbol{\beta}})}{n-m-1}\right] = \sigma^2$. Thus, $\hat{\sigma}^2 = \frac{S(\hat{\boldsymbol{\beta}})}{n-m-1}$ is an unbiased estimator of σ^2, making $\hat{\sigma} = \sqrt{\hat{\sigma}^2}$ a reasonable point estimate for σ. From the Bayesian perspective, the posterior mean of σ^2 is $\frac{S(\hat{\boldsymbol{\beta}})}{n-m-3}$ (for $n - m - 1 > 2$), which approximates $\hat{\sigma}^2$ when $n - m - 1$ is large.